

# W7D4 Report

## Obiettivo

L'obiettivo dell'esercitazione è acquisire i fondamenti LLM attraverso il linguaggio Python, con particolare riferimento al loro utilizzo nell'ambito della Cyber Security.

L'attività prevede inizialmente la configurazione di un ambiente Python isolato e l'integrazione con le Gemini API mediante la libreria google-genai. Successivamente verrà realizzato un semplice script denominato Hello Gemini, finalizzato a comprendere il processo di invio di un prompt e di ricezione della risposta .

Nella seconda parte dell'esercitazione verrà sviluppato un Security Assistant interattivo, configurato attraverso un ciclo di conversazione.

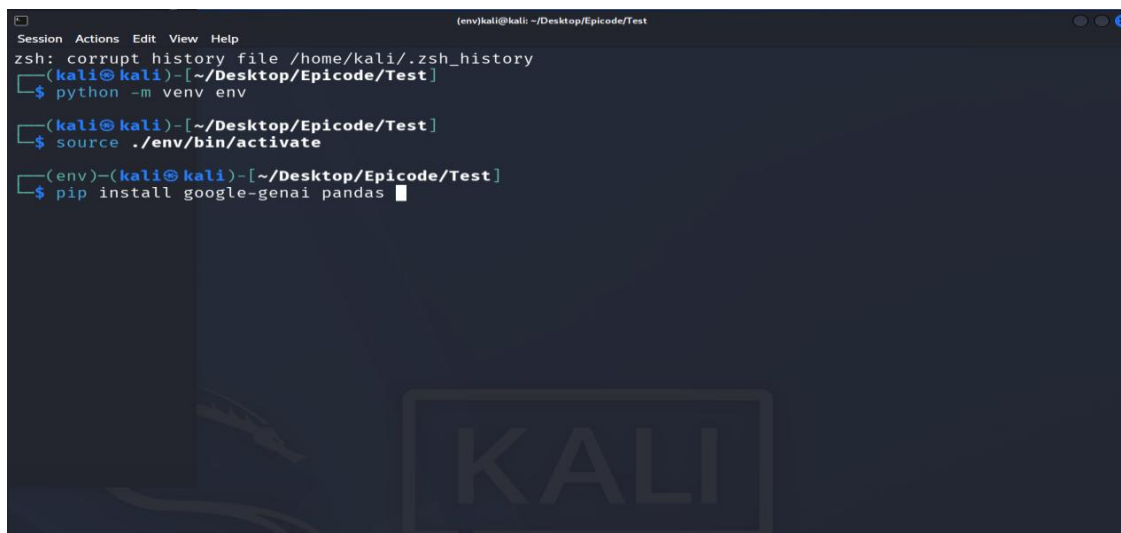
## Setup dell'ambiente virtuale + Gemini API

Per lo svolgimento dell'esercitazione è stata innanzitutto creata una nuova cartella di lavoro, separata dalle attività precedenti, all'interno della quale è stato configurato un ambiente virtuale Python.

L'ambiente virtuale è stato creato utilizzando il comando:

```
python -m venv env
```

L'utilizzo di un *virtual environment* permette di mantenere isolate le librerie e le dipendenze necessarie al progetto rispetto a quelle installate globalmente nel sistema, riducendo il rischio di conflitti tra versioni differenti dei pacchetti. Successivamente, l'ambiente virtuale è stato attivato tramite il comando: `source ./env/bin/activate`



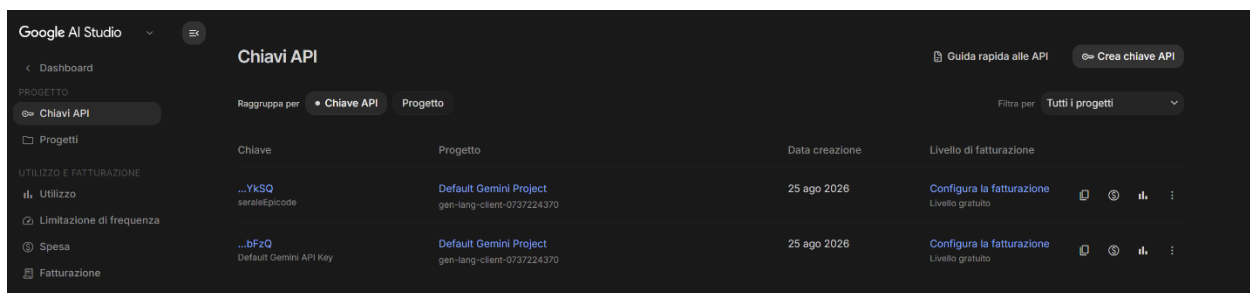
```
(env)kali@kali: ~/Desktop/Epicode/Test
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~/Desktop/Epicode/Test]
└─$ python -m venv env
(kali@kali)-[~/Desktop/Epicode/Test]
└─$ source ./env/bin/activate
(env)-(kali@kali)-[~/Desktop/Epicode/Test]
└─$ pip install google-genai pandas
```

Una volta attivato l'ambiente, sono state installate le librerie necessarie allo svolgimento dell'esercitazione mediante:

```
pip install google-genai pandas python-dotenv
```

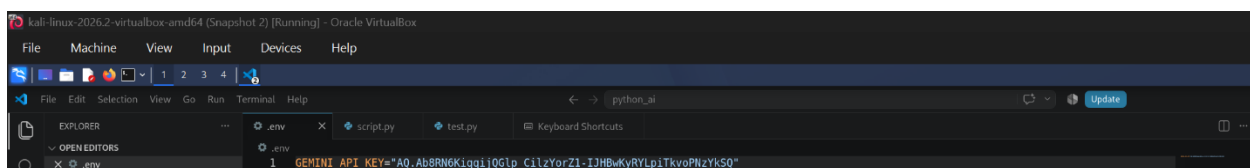
Tra le librerie installate, google-genai consente di interagire con i modelli Gemini attraverso le relative API, mentre pandas fornisce funzionalità per la gestione e l'analisi dei dati. La libreria python-dotenv viene invece utilizzata per la gestione delle variabili d'ambiente, permettendo di caricare configurazioni e credenziali da un file `.env` senza inserirle direttamente all'interno del codice Python.

Successivamente è stata configurata la chiave API necessaria per consentire allo script Python di comunicare con i modelli Gemini. Dopo aver generato la propria API Key, è stato creato un file denominato `.env` all'interno della cartella del progetto.



All'interno del file `.env` è stata inserita la chiave API associandola a una variabile d'ambiente, ad esempio:

```
GEMINI_API_KEY="la-tua-api-key"
```



L'utilizzo del file `.env` consente di mantenere le informazioni sensibili, come le chiavi di accesso alle API, separate dal codice sorgente. In questo modo si evita di inserire direttamente la chiave all'interno dello script Python.

La libreria python-dotenv, precedentemente installata, verrà utilizzata successivamente per caricare la variabile contenuta nel file `.env` all'interno del script creato.

## Parte I – Script “Hello Gemini”

Dopo aver configurato l'ambiente virtuale e la chiave API, è stato realizzato un primo script con l'obiettivo di verificare la corretta comunicazione tra Python e il modello Gemini. L'esercizio consiste nell'inviare un semplice messaggio al modello e visualizzare successivamente la risposta generata.

Per prima cosa sono state importate le librerie necessarie:

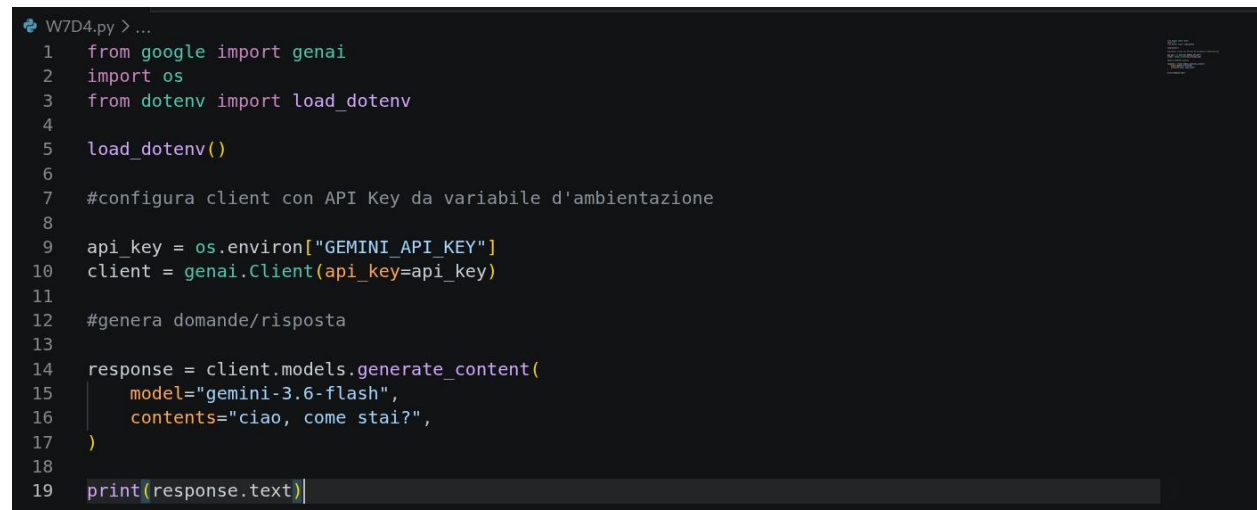
```
from google import genai
```

```
import os
```

```
from dotenv import load_dotenv
```

La libreria google-genai permette di utilizzare le API di Gemini attraverso Python, mentre il modulo os viene utilizzato per accedere alle variabili d'ambiente. La funzione load\_dotenv(), fornita dalla libreria python-dotenv, consente invece di caricare all'interno dell'ambiente Python le variabili definite nel file .env.

```
load_dotenv()
```



```
W7D4.py > ...
1  from google import genai
2  import os
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  #configura client con API Key da variabile d'ambientazione
8
9  api_key = os.environ["GEMINI_API_KEY"]
10 client = genai.Client(api_key=api_key)
11
12 #genera domande/risposta
13
14 response = client.models.generate_content(
15     model="gemini-3.6-flash",
16     contents="ciao, come stai?",
17 )
18
19 print(response.text)
```

```
api_key = os.environ["GEMINI_API_KEY"]
```

```
client = genai.Client(api_key=api_key)
```

In questa fase viene recuperata la variabile GEMINI\_API\_KEY contenuta nel file .env e utilizzata per configurare il client. In questo modo la chiave API non viene inserita direttamente all'interno del codice sorgente, ma viene richiamata in modo sicuro attraverso una variabile d'ambiente.

Successivamente è stata effettuata la richiesta al modello Gemini tramite il metodo generate\_content(). In particolare, è stato specificato il modello gemini-3.6-flash e come contenuto della richiesta è stato utilizzato il semplice messaggio "ciao, come stai?".

```
response = client.models.generate_content(
```

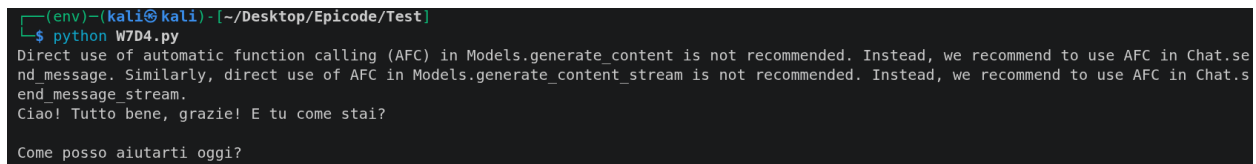
```
    model="gemini-3.6-flash",
```

```
    contents="ciao, come stai?",  
)
```

Infine, la risposta restituita dal modello è stata visualizzata nel terminale attraverso:

```
print(response.text)
```

L'esecuzione dello script permette quindi di verificare il corretto funzionamento dell'ambiente configurato e della comunicazione con le API di Gemini, costituendo il primo esempio di interazione tra un programma Python e un assistant AI



```
(env)-(kali@kali) - [~/Desktop/Epicode/Test]  
$ python W7D4.py  
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.  
Ciao! Tutto bene, grazie! E tu come stai?  
  
Come posso aiutarti oggi?
```

## Parte II -Chatbot Security Assistant interattivo

Dopo aver verificato il corretto funzionamento della comunicazione tra Python e Gemini, il codice precedente è stato modificato per trasformare il semplice script Hello Gemini in un Security Assistant

La principale modifica introdotta riguarda l'utilizzo del parametro `system_instruction`. Questa istruzione permette di definire il ruolo, il comportamento e le modalità di risposta che il modello dovrà adottare durante l'interazione.

Nel caso specifico, il modello è stato configurato come un assistente specializzato in Cyber Security:

```
config = types.GenerateContentConfig(  
    system_instruction="""Sei un esperto di CyberSecurity e analizzi log da anni.
```

```
  
    Rispondi sempre in maniera chiara e precisa.
```

```
  
    Se non sei sicuro dillo, ma non inventarlo mai.""",  
    temperature=0.2,
```

La system instruction stabilisce quindi il contesto operativo dell'assistente, indirizzandolo verso l'analisi di problematiche relative alla sicurezza informatica. È stato inoltre specificato che le risposte devono essere chiare e precise e che, in caso di incertezza, il modello deve dichiararla esplicitamente evitando di fornire informazioni non verificate.

```

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
from google.genai import types
config = types.GenerateContentConfig(
    system_instruction=' Rispondi in modo semplice e chiaro. '
)

#esempi per rimuovere le censure
safety_settings = [
    types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
    types.SafetySetting(category="HARM_CATEGORY_SEXUAL_EXPLICIT", threshold="BLOCK_NONE"),
]

config=types.GenerateContentConfig(
    #system instruction definiremo il ruolo del nostro AI
    system_instruction="" Sei un esperto di CyberSecurity e analizzi log da anni
    rispondi sempre in maniera chiara e precisa.
    Se non sei sicuro dillo, ma non inventarlo mai"",
    temperature=0.2,
)

```

Per quanto riguarda le impostazioni di sicurezza, sono state definite specifiche SafetySetting attraverso il modulo types, associando alle diverse categorie una soglia di intervento configurabile. Questa impostazione permette di gestire il comportamento dei filtri di sicurezza del modello in relazione ai contenuti elaborati dall'assistente.

L'introduzione della system instruction rappresenta quindi un'evoluzione rispetto al precedente esercizio: mentre nel programma Hello Gemini il modello rispondeva semplicemente al prompt ricevuto, in questo caso viene definito preventivamente un ruolo specifico, consentendo di ottenere risposte maggiormente orientate al contesto della Cyber Security.

Infine, lo script è stato modificato per trasformare l'interazione con Gemini in una conversazione continua. È stato utilizzato un ciclo while True, che permette al programma di continuare a chiedere domande all'utente e a visualizzare le relative risposte.

Il comando break viene utilizzato per interrompere il ciclo e terminare il programma quando richiesto dall'utente tramite il testo “esci”

In questo modo, il Security Assistant non risponde più a una sola domanda, ma permette di instaurare una conversazione interattiva.

```
# loop per conversazione continua
while True:
    domanda = input("Tu: ")
    if domanda.lower() == "esci":
        print("Arrivederci!")
        break

    response = client.models.generate_content(
        model="gemini-3.6-flash",
        contents=domanda,
        config=config
    )
    print(f"AI: {response.text}\n")
```

Per concludere l'esercitazione, è stato chiesto al Security Assistant di spiegare brevemente che cosa sia un attacco SQL Injection, verificando così la capacità di fornire una risposta pertinente a una domanda di Cyber Security.

```
(env)-(kali@kali)-[~/Desktop/Epicode/Test]
└─$ python WZD4.py
/home/kali/Desktop/Epicode/Test/env/lib/python3.13/site-packages/google/genai/_common.py:668: UserWarning: HARM_CATEGORY_SEXUAL_EXPLICIT is not a valid HarmCategory
  warnings.warn(f'{value} is not a valid {cls.__name__}')
Tu: Ciao, Come va?
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
AI: Ciao! Tutto bene, grazie.

Sono pronto ad aiutarti. Che si tratti di analizzare log di sistema, identificare potenziali minacce, configurare SIEM o rispondere a dubbi sulla sicurezza informatica, sono a tua disposizione.

Come posso esserti utile oggi? Hai dei log da esaminare o un caso specifico da analizzare?
Tu: Vorrei che mi dicessi in 3 semplici frasi cos'e' un attacco sql
AI: Un attacco SQL (o SQL Injection) è una tecnica in cui un malintenzionato inserisce codice malevolo all'interno dei campi di input di un sito web, come un modulo di login o una barra di ricerca.
L'applicazione invia erratamente questo codice al proprio database, ingannandolo e facendogli eseguire comandi non previsti dagli sviluppatori.
In questo modo, l'hacker può aggirare le protezioni di sicurezza per visualizzare, modificare o eliminare dati riservati custoditi nel sistema.

Tu: esci
Arrivederci!
```